Roles/Users grants and permission Runbook

Intro

PostgreSQL uses the concept of roles to manage database access. The roles are global objects which means that a role doesn't belong to a specific database but all of them and can access all databases if given the appropriate permissions.

The term of `roles` encapsulates the concepts of groups and users at the same time. In practice, what differences a more abstract "role" with a "user" is the set of `attributes` it has. In this example, `LOGIN` and `NOLOGIN` are attributes a role can have.

- user: can login
- group: can't login

In the following sections, we will see some common activities to manage roles and user in PostgreSQL

Note: To do some of the following operations you will need to have `superuser` or `create role` attributes or be owner of the objects.

Creating new roles and users

To create roles/user you can use the command `CREATE ROLE`

You also can use the command: `CREATE USER`, it is an alias for CREATE ROLE + LOGIN clause

create user

```
CREATE USER user1 WITH password 'pass1';
```

Create a group

```
CREATE GROUP administrators SUPERUSER;
```

And add users to that group

```
ALTER GROUP administrators add user user1;
```

create role

```
CREATE role readonly_role;
```

And give that role to a user

```
GRANT readony_role to user1;
```

To check the created Roles/Users in the database instance, you can use the meta-commands `\dg` or `\du`

```
postgres=# \dg

                                   List of roles
    Role name    |                         Attributes                         | Member of
-----------------+------------------------------------------------------------+-----------
 postgres        | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
 readonly_roles  | Cannot login                                               | {}
 user1           |                                                            | {}
```

The most common rights for Roles/Users

To define access privileges to roles/users, you must use the command `GRANT`

On tables

USAGE: Permission of SELECT/INSERT/UPDATE/DELETE/TRUNCATE/REFERENCES/TRIGGER

```
GRANT SELECT on new_schema.new_table to user1;
```

```
GRANT INSERT on new_schema.new_table to user1;
```

```
GRANT UPDATE, DELETE on new_schema.new_table to user1;
```

You can use the clause `ALL PRIVILEGES` to grant all permissions at once.

```
GRANT ALL PRIVILEGES on new_schema.new_table to user1;
```

If we want to grant permission on all the tables of a specific schema we can use `ALL TABLES IN SCHEMA` clause:

```
GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA new_schema TO user1;
```

In schemas

USAGE: Permission of usage

```
GRANT USAGE on SCHEMA new_schema to user1;
```

CREATE: Permission of create objects

```
GRANT CREATE on SCHEMA new_schema to user1;
```

There are another types of object to granting permission, for example, SEQUENCE, FUNCTIONS, DOMAIN, etc

In Roles

You also can grant permission from `role/user` to another `role/user`:

```
postgres=# GRANT readonly_roles to user1 ;
GRANT ROLE
postgres=# \dg
                                   List of roles
```

```
    Role name    |                         Attributes                         | Member of
-----------------+------------------------------------------------------------+-----------
 postgres        | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
 readonly_roles  | Cannot login                                               | {}
 user1           |                                                            | {readonly_ro
--user1 will INHERIT permissions from readonly_roles
grant SELECT on ALL tables in schema public to readonly_roles ;
```

Verify the access of roles/users

```
select grantor,grantee,table_schema||'.'||table_name as table, string_agg(privilege_type,',
     grantor      |    grantee     |                          table
------------------+----------------+-----------------------------------------------------
 gitlab           | readonly_roles | public.abuse_reports
 gitlab           | readonly_roles | public.alert_management_alert_assignees
 gitlab           | readonly_roles | public.alert_management_alert_user_mentions
 gitlab           | readonly_roles | public.alert_management_alerts
...
```

Permissions and User Defined Functions

If a non-privleged user needs to execute a function that access privileged data, the `SECURITY DEFINER` clause can be used when creating a funcion:

(Using a privileged user):

```
CREATE FUNCTION function_name(...)
SECURITY DEFINER
AS
$$
SELECT * from some_table...
$$
...
```

This way the function is executed with the privileges of the user who created it, so other users can execute the function even if they don't have access to the underlying objects it might access.

Default and implicit roles

PostgreSQL has default roles, molstly related to statistical and administrative access. You can find more information here. In addition, there is a default superuser, generally called `postgres`.

There is also an "implicit" group, called `PUBLIC` that refers to every role (including those that will be created later) that can be used when granting/revoking permissions:

```
GRANT SELECT ON some_table to PUBLIC;
```

Revoke permission

To revoke access privileges from roles/users, you must use the command `REVOKE`

```
REVOKE SELECT on all tables in schema public from readonly_roles ;
```

To revoke a role

```
REVOKE administrators from user1;
```

Decommission a user

A roles/users can be deleted from the database using commnad `DROP ROLE`, make sure the user doesn't have permission dependencies

```
DROP ROLE readonly_roles ;
```

Modify pg_hba conf

PostgreSQL manages client authorization using a configuration file called `pg_hba.conf` and sometimes it is required to adjust this file for access rights - if you don't have permission to connect, you will see an error similar to:

```
connect to PostgreSQL server: FATAL: no pg_hba.conf entry for host "XXX.XXX.XX.XXX", user "u
```

You must fix it adding a row for the user in the `pg_hba.conf` file, example:

```
# TYPE  DATABASE    USER        CIDR-ADDRESS          METHOD
host    dbXXX         userXXX         XXX.XXX.XX.XXX/N_mask_byte     md5
```

Important: Gitlab uses a Patroni managed cluster, in that deployment mode the correct method to add manage client authorization is through the `pg_hba` settings defined in the `patroni.yml` file. Any entries persisted directly into the `pg_hba.conf` can be overriden by Patroni.

Propagated user to PGBouncer

PGBouncer also needs to be setup for authenticating users. In the simplest case, pgBouncer uses its own file for storing users and passwords (by default `userlist.txt`). But pgBouncer can also query the database to authenticate the user being connected to pgBouncer. In our case we are doing it via auth_query parameter, like in:

```
auth_query = SELECT username, password FROM public.pg_shadow_lookup($1)
```

This is another example of a function with `SECURITY DEFINER` clause. Since accessing `pg_shadow` (where user passwords resides) needs admin rights, it is better to use a non-superuser calling a `SECURITY DEFINER` function:

```
CREATE OR REPLACE FUNCTION public.pg_shadow_lookup(i_username text, OUT username text, OUT p
 RETURNS record
```

```
 LANGUAGE plpgsql
 SECURITY DEFINER
AS $function$
BEGIN
    SELECT usename, passwd FROM pg_catalog.pg_shadow
    WHERE usename = i_username INTO username, password;
    RETURN;
END;
$function$
```

Dumping roles from the database

A common mistakes when restoring a database with a backup using `pg_dump` program, is that `pg_dump` does not include the necessary commands for restoring users, leading to errors when re-creating objects permisions:

```
pg_restore -f schema.backup -d new_database
```

```
pg_restore: [archiver (db)] Error while PROCESSING TOC:
pg_restore: [archiver (db)] Error from TOC entry 3969; 0 0 ACL public gitlab
pg_restore: [archiver (db)] could not execute query: ERROR:  role "gitlab" does not exist
```

In order to avoid those errors, you must create all roles previously:

- First, (in the "origin") export them with `pg_dumpall --roles-only > backup-roles.sql` . That will create an SQL file that you can feed to a new postgres instance,
- In the destination, create the roles with `psql < backup-roles.sql`

Giving permissions to objects that will be created in the future

When a new table is created, by default only the table creator and the superusers can access it. That behaviour can be changed using `ALTER DEFAULT PRIVILEGES`, like in

```
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO analytics;
```

Configuration management of database users

When a new empty PostgreSQL/Patroni cluster is deployed it creates any `user` as defined in the `patroni.yml` file, in the `bootstrap.users` section, as documented at: https://patroni.readthedocs.io/en/latest/SETTINGS.html#bootstrap-configuration

If you are launching a new Patroni environment using Gitlab's `chef-repo`, then the database users can be defined under `['gitlab-patroni']['patroni']['users']`, which will then be configured by a proper recipe into the proper `patroni.yml`

section, as explained at https://gitlab.com/gitlab-com/gl-infra/reliability/-/issues/15212#note_845820648

The default users can be defined using 2 different methods:

- The default users created are defined in the `gitlab-patroni` cookbook under `attributes/default.rb` - https://gitlab.com/gitlab-cookbooks/gitlab-patroni/-/blob/master/attributes/default.rb#L53
- But for custom deployments they can also be defined in the role definition, eg. https://gitlab.com/gitlab-com/gl-infra/chef-repo/-/blob/master/roles/gstg-base-db-patroni.json#L50

Important: the above method is through `bootstrap`, which means that only create users into a new Patroni cluster, therefore the method will not work to create new users into an existing Patroni cluster*

Note: Gitlab's `gstg` and `gprd` databases have several users that were created through deployment or migration, hence they are not defined in our `chef-repo` repository.

If your users needs any privilege provided only by `GRANT`, then you should write your own deployment script and execute on each new environment, because the Patroni `boostrap.users.options` setting only accept options of CREATE USER statement